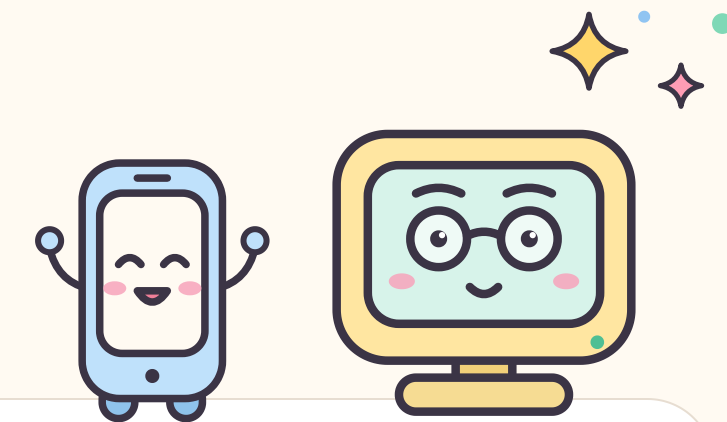




第 7 集 · 约 23 分钟

WASM②： JS 胶水与测试

一百多行 JavaScript，让浏览器用上 Rust

配套阅读：书的第 3 章



从零写出 LanShare

这一集你会学到

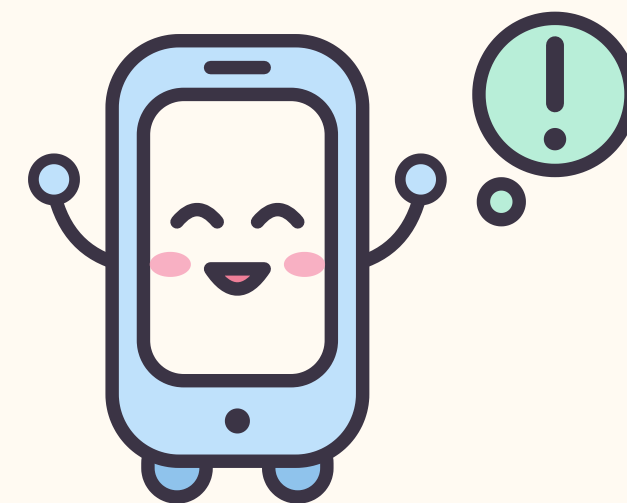
1. loadCrypto：加载 WASM，把随机数函数递给它
2. call()：申请内存 → 拷入 → 调用 → 拷出 → 释放，一处写对，处处不漏
3. 两个 JS 坑：内存扩容后旧视图失效，64 位计数器要拆开传
4. 用同一份已知向量测 WASM：OpenSSL、原生 Rust、WASM 三方比对
5. lanshare.wasm：怎么编出来，为什么和仓库里那份不一样



先想一个问题

JS 手里有一个 1 MiB 的文件块，要交给 Rust 加密。最少要调几次 WASM 函数？

9 次：4 次 `ls_alloc`、1 次 `ls_seal`、4 次 `ls_free`。密钥、附加数据、明文、输出，每块内存都要一借一还。





proto.js：文件说明

● ● ●

proto.js crates/lanshare/web/

第 1-5 行 • 共 5 行

```
1 // LanShare 浏览器端协议客户端。
2 // 加解密全部在 lanshare.wasm 里完成（与服务端是同一份 Rust 代码）；
3 // 本文件负责：把字节搬进/搬出 WASM 线性内存、配对、加密请求。
4
5 const encoder = new TextEncoder();
```



loadCrypto: 准备导入对象

proto.js

crates/lanshare/web/

第 1-20 行 • 共 20 行

```

1 // LanShare 浏览器端协议客户端。
2 // 加解密全部在 lanshare.wasm 里完成 (与服务端是同一份 Rust 代码) ;
3 // 本文件负责: 把字节搬进/搬出 WASM 线性内存、配对、加密请求。
4
5 const encoder = new TextEncoder();
6
7 /** 把 Uint8Array / ArrayBuffer / URL 加载成加密工具集。 */
8 export async function loadCrypto(source, getRandomValues = (buf) => globalThis.crypto.getRandomValues(buf)) {
9     let memory;
10     const imports = {
11         env: {
12             // WASM 需要随机数时调用; getRandomValues 单次最多 65536 字节, 分段取
13             ls_random(ptr, len) {
14                 const target = new Uint8Array(memory.buffer, ptr >>> 0, len >>> 0);
15                 for (let off = 0; off < target.length; off += 65536) {
16                     getRandomValues(target.subarray(off, Math.min(off + 65536, target.length)));
17                 }
18             },
19         },
20     };

```



加载并实例化 WASM

proto.js crates/lanshare/web/

第 10-29 行 • 共 29 行

```

10  const imports = {
11    env: {
12      // WASM 需要随机数时调用; getRandomValues 单次最多 65536 字节, 分段取
13      ls_random(ptr, len) {
14        const target = new Uint8Array(memory.buffer, ptr >>> 0, len >>> 0);
15        for (let off = 0; off < target.length; off += 65536) {
16          getRandomValues(target.subarray(off, Math.min(off + 65536, target.length)));
17        }
18      },
19    },
20  };
21  let bytes = source;
22  if (!(source instanceof Uint8Array || source instanceof ArrayBuffer)) {
23    const response = await fetch(source);
24    if (!response.ok) throw new Error("加载加密模块失败 (" + response.status + ") ");
25    bytes = await response.arrayBuffer();
26  }
27  const { instance } = await WebAssembly.instantiate(bytes, imports);
28  const x = instance.exports;
29  memory = x.memory;

```



最重要的两个函数: view 和 call

proto.js crates/lanshare/web/

第 26-45 行 • 共 45 行

```

26  }
27  const { instance } = await WebAssembly.instantiate(bytes, imports);
28  const x = instance.exports;
29  memory = x.memory;
30
31  // 线性内存增长后旧的 ArrayBuffer 会失效, 所以每次都现取视图
32  const view = (ptr, len) => new Uint8Array(memory.buffer, ptr, len);
33
34  /** 申请输入/输出缓冲区 → 拷入输入 → 调用 → 拷出输出 → 全部释放。 */
35  function call(inputs, outLens, fn) {
36      const ins = inputs.map((data) => ({ data, len: data.length, ptr: x.ls_alloc(data.length) >>> 0 }));
37      const outs = outLens.map((len) => ({ len, ptr: x.ls_alloc(len) >>> 0 }));
38      try {
39          for (const b of ins) if (b.len) view(b.ptr, b.len).set(b.data);
40          const ret = fn(...ins.map((b) => b.ptr), ...outs.map((b) => b.ptr));
41          return { ret, outs: outs.map((b) => view(b.ptr, b.len).slice()) };
42      } finally {
43          for (const b of [...ins, ...outs]) x.ls_free(b.ptr, b.len);
44      }
45  }

```



跨语言调用: Python 的 ctypes 也是这样

Python • ctypes 调 C 动态库 (示意)

```
1 import ctypes
2 lib = ctypes.CDLL("crypto.dll")
3 buf = ctypes.create_string_buffer(1024) # 申请一块 C 能读写的内存
4 lib.seal(key, ctr_hi, ctr_lo, buf, len(buf)) # 只能传指针和数字
5 result = buf.raw # 把结果拷回 Python
```

JS 调 WASM (简化)

```
1 const ptr = x.ls_alloc(len); // 申请一块 WASM 能读写的内存
2 view(ptr, len).set(data); // 把数据写进去
3 x.ls_seal(k, hi, lo, a, aadLen, ptr, len, out); // 只能传指针和数字
4 const result = view(out, len + 16).slice(); // 把结果拷出来
5 x.ls_free(ptr, len);
```



64 位计数器拆成两半



proto.js

crates/lanshare/web/

第 32-51 行 • 共 51 行

```
32  const view = (ptr, len) => new Uint8Array(memory.buffer, ptr, len);
33
34  /** 申请输入/输出缓冲区 → 拷入输入 → 调用 → 拷出输出 → 全部释放。 */
35  function call(inputs, outLens, fn) {
36    const ins = inputs.map((data) => ({ data, len: data.length, ptr: x.ls_alloc(data.length) >>> 0 }));
37    const outs = outLens.map((len) => ({ len, ptr: x.ls_alloc(len) >>> 0 }));
38    try {
39      for (const b of ins) if (b.len) view(b.ptr, b.len).set(b.data);
40      const ret = fn(...ins.map((b) => b.ptr), ...outs.map((b) => b.ptr));
41      return { ret, outs: outs.map((b) => view(b.ptr, b.len).slice()) };
42    } finally {
43      for (const b of [...ins, ...outs]) x.ls_free(b.ptr, b.len);
44    }
45  }
46
47  const split = (ctr) => {
48    const big = BigInt(ctr);
49    return [Number((big >> 32n) & 0xffffffffn), Number(big & 0xffffffffn)];
50  };
51  const bytesOf = (s) => (typeof s === "string" ? encoder.encode(s) : s);
```



对外的接口：加密和解密

proto.js crates/lanshare/web/

第 46–65 行 • 共 65 行

```

46
47  const split = (ctr) => {
48      const big = BigInt(ctr);
49      return [Number((big >> 32n) & 0xfffffffffn), Number(big & 0xfffffffffn)];
50  };
51  const bytesOf = (s) => (typeof s === "string" ? encoder.encode(s) : s);
52
53  return {
54      seal(key, ctr, aad, plain) {
55          const [hi, lo] = split(ctr);
56          return call([key, aad, plain], [plain.length + 16], (k, a, p, out) =>
57              x.ls_seal(k, hi, lo, a, aad.length, p, plain.length, out)).outs[0];
58      },
59      open(key, ctr, aad, sealed) {
60          if (sealed.length < 16) return null;
61          const [hi, lo] = split(ctr);
62          const r = call([key, aad, sealed], [sealed.length - 16], (k, a, s, out) =>
63              x.ls_open(k, hi, lo, a, aad.length, s, sealed.length, out));
64          return r.ret === 0 ? r.outs[0] : null;
65      },

```



握手和配对的接口

proto.js crates/lanshare/web/

第 89-108 行 • 共 108 行

```

89     pakeStart(pin, serverId) {
90         const p = bytesOf(pin), s = bytesOf(serverId);
91         const r = call([p, s], [x.ls_pake_msg_len()], (a, b, out) => x.ls_pake_start(a, p.length, b, s.length, out));
92         if (r.ret === 0) throw new Error("口令或服务器标识不是有效的 UTF-8");
93         return { handle: r.ret, msg: r.outs[0] };
94     },
95     pakeFinish(handle, peerMsg) {
96         const r = call([peerMsg], [32], (a, out) => x.ls_pake_finish(handle, a, peerMsg.length, out));
97         return r.ret === 0 ? r.outs[0] : null;
98     },
99     randomBytes(n) {
100         const buf = new Uint8Array(n);
101         for (let off = 0; off < n; off += 65536) getRandomValues(buf.subarray(off, Math.min(off + 65536, n)));
102         return buf;
103     },
104     memoryBytes() {
105         return memory.buffer.byteLength;
106     },
107 };
108 }

```



测试: WASM 对已知向量



crypto.test.mjs

web-tests/

第 73-92 行 • 共 92 行

```

73 test("内存不泄漏: 反复加解密后线性内存不再增长", () => {
74   const k = crypto.randomBytes(32), plain = crypto.randomBytes(1 << 20);
75   for (let i = 0; i < 20; i++) crypto.open(k, 1n, enc("a"), crypto.seal(k, 1n, enc("a"), plain));
76   const before = crypto.memoryBytes();
77   for (let i = 0; i < 200; i++) crypto.open(k, 1n, enc("a"), crypto.seal(k, 1n, enc("a"), plain));
78   assert.equal(crypto.memoryBytes(), before);
79 });
80
81 test("测速 (记入 docs/bench.md) ", () => {
82   const k = crypto.randomBytes(32), plain = crypto.randomBytes(1 << 20), aad = enc("lanshare/2 req /api/up/x/0");
83   const N = 256;
84   let t = performance.now(), sealed;
85   for (let i = 0; i < N; i++) sealed = crypto.seal(k, BigInt(i + 1), aad, plain);
86   const sealMiBs = N / ((performance.now() - t) / 1000);
87   t = performance.now();
88   for (let i = 0; i < N; i++) crypto.open(k, BigInt(N), aad, sealed);
89   const openMiBs = N / ((performance.now() - t) / 1000);
90   console.log(`WASM seal ${sealMiBs.toFixed(0)} MiB/s, open ${openMiBs.toFixed(0)} MiB/s (Node ${process.version}, 1 MiB chunks)`);
91   assert.ok(sealMiBs > 50 && openMiBs > 50);
92 });

```





lanshare.wasm：放进网页目录

lanshare.wasm

crates/lanshare/web/从仓库拷贝

二进制文件，直接从仓库拷贝



自己编的，和仓库里的一样吗？

```
Windows PowerShell

PS D:\code\lanshare>
(Get-FileHash target\wasm32-unknown-unknown\release\lanshare_wasm.wasm, crates\lanshare\web\lanshare.wasm).Hash
FB480907FBF588AC258989E3054984B4A5A4117D91FEB4248D21393AF45AE96A
86AC197905AB6C7102F44F8E18543D85BF356516A7B4FBD1E1BD0BCB78313E5F

PS D:\code\lanshare>
[regex]::Matches((Get-Content target\wasm32-unknown-unknown\release\lanshare_wasm.wasm -Raw),
'[\w:\\/.-]*registry[\w:\\/.-]*').Value
C:\Users\<你>\.cargo\registry\src\index.crates.io-1949cf8c6b5b557f\rand_core-0.6.4\src\os.rs
C:\Users\<你>\.cargo\registry\src\index.crates.io-1949cf8c6b5b557f\cipher-0.4.4\src\stream.rs
C:\Users\<你>\.cargo\registry\src\index.crates.io-1949cf8c6b5b557f\spake2-0.4.0\src\ed25519.rs

PS D:\code\lanshare>
[regex]::Matches((Get-Content crates\lanshare\web\lanshare.wasm -Raw), '[\w:\\/.-]*registry[\w:\\/.-]*').Value
/cargo\registry\src\index.crates.io-1949cf8c6b5b557f\rand_core-0.6.4\src\os.rs
/cargo\registry\src\index.crates.io-1949cf8c6b5b557f\cipher-0.4.4\src\stream.rs
/cargo\registry\src\index.crates.io-1949cf8c6b5b557f\spake2-0.4.0\src\ed25519.rs
```





为什么 WASM 里有你的用户名

- 出错时要报“哪个文件第几行”，编译器把源文件路径写进了产物
- 自己的代码是相对路径；依赖库在 cargo 的下载目录里，是绝对路径
- 发出去的文件，等于公开了你的用户名
- 发布用 `build.ps1: --remap-path-prefix` 把 cargo 目录换成 `/cargo`



拷进 web 目录，跑测试

```
Windows PowerShell

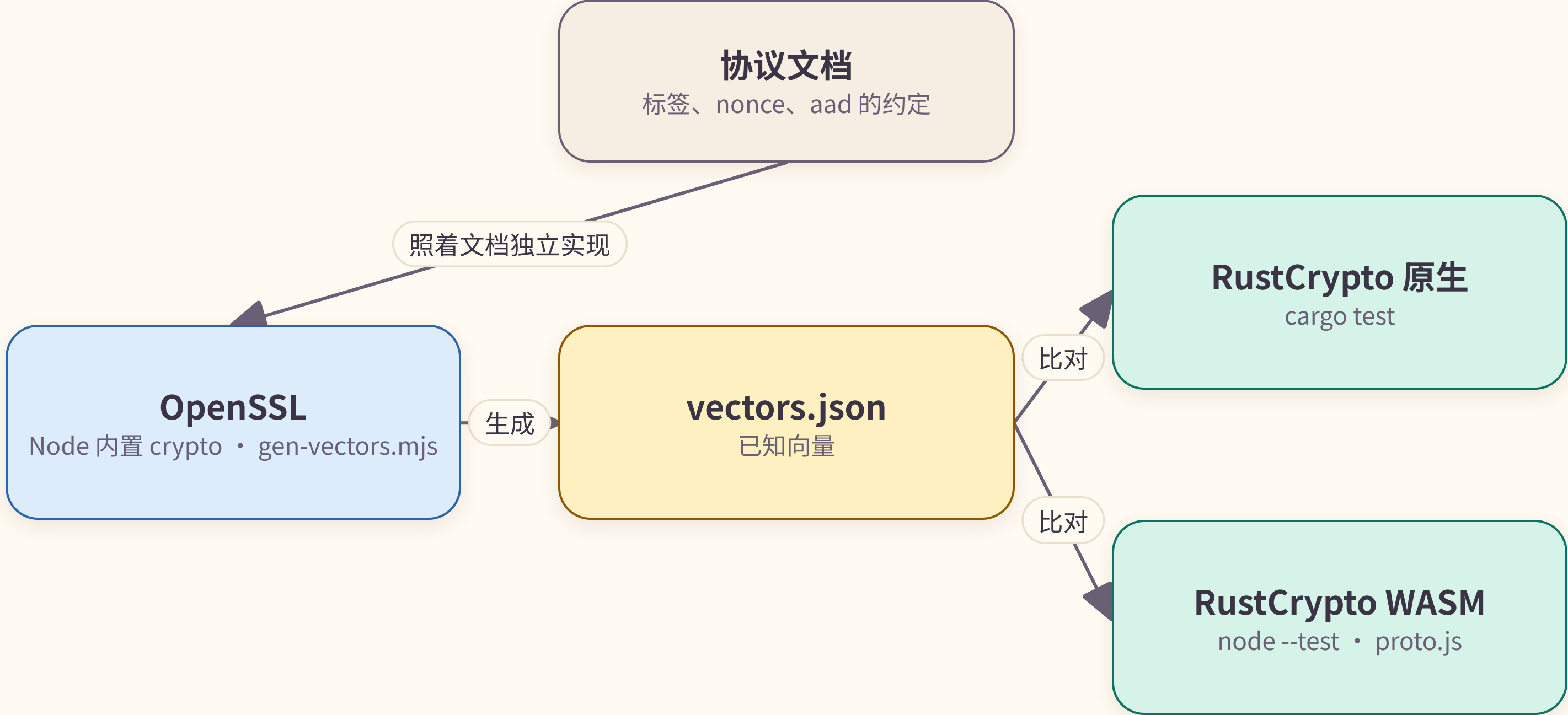
PS D:\code\lanshare>
Copy-Item target\wasm32-unknown-unknown\release\lanshare_wasm.wasm crates\lanshare\web\lanshare.wasm -Force

PS D:\code\lanshare> node --test web-tests/crypto.test.mjs
WASM seal 364 MiB/s, open 363 MiB/s (Node v24.12.0, 1 MiB chunks)
✓ 握手标签、会话密钥、方向密钥与 OpenSSL 一致 (1.6261ms)
✓ 确认标签与 OpenSSL 一致，校验区分角色 (0.5319ms)
✓ 加密结果与 OpenSSL 逐字节一致 (含 64 位计数器) (0.403ms)
✓ 解密：正确的能解开，任何篡改都解不开 (0.2854ms)
✓ 大块数据往返 (1 MiB + 3) (13.5193ms)
✓ PAKE：同口令同钥匙，错口令不同钥匙，句柄只能用一次 (3.1025ms)
✓ 内存不泄漏：反复加解密后线性内存不再增长 (1253.0573ms)
✓ 测速 (记入 docs/bench.md) (1412.0728ms)
i tests 8
i suites 0
i pass 8
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 2784.4703
```





三方比对，两层证据





实验：忘了释放内存会怎样

```
Windows PowerShell

PS D:\code\lanshare> node --test web-tests/crypto.test.mjs
WASM seal 85 MiB/s, open 59 MiB/s (Node v24.12.0, 1 MiB chunks)
✓ 握手标签、会话密钥、方向密钥与 OpenSSL 一致 (1.7556ms)
✓ 确认标签与 OpenSSL 一致, 校验区分角色 (0.5917ms)
✓ 加密结果与 OpenSSL 逐字节一致 (含 64 位计数器) (0.515ms)
✓ 解密: 正确的能解开, 任何篡改都解不开 (0.3204ms)
✓ 大块数据往返 (1 MiB + 3) (15.3067ms)
✓ PAKE: 同口令同钥匙, 错口令不同钥匙, 句柄只能用一次 (3.7742ms)
✗ 内存不泄漏: 反复加解密后线性内存不再增长 (3024.5138ms)
✓ 测速 (记入 docs/bench.md) (7386.3215ms)
i tests 8
i suites 0
i pass 7
i fail 1
i cancelled 0
i skipped 0
i todo 0
i duration_ms 10656.2581

✗ failing tests:
```



选择题

proto.js 为什么不把 `new Uint8Array(memory.buffer)` 存起来反复用，而是每次都现开一个视图？

- A 为了省内存
- B 线性内存扩容后，旧的 `ArrayBuffer` 会失效，存起来的视图就读不到数据了
- C 因为 JS 不允许保存视图



选择题

proto.js 为什么不把 `new Uint8Array(memory.buffer)` 存起来反复用，而是每次都现开一个视图？

A 为了省内存

开视图本身几乎不占内存，和省内存无关。

C 因为 JS 不允许保存视图

JS 允许，只是保存下来的视图在扩容后会失效。

B 线性内存扩容后，旧的 `ArrayBuffer` 会失效，存起来的视图就读不到数据了

扩容后旧的 `ArrayBuffer` 被分离，旧视图长度变成 0。每次从当前的 `memory.buffer` 现开，才不会出错。



选择题

call 函数里，释放内存为什么写在 finally 里？

- A 因为 finally 里的代码跑得更快
- B 不管调用成功还是中途抛出异常，申请的内存都必须还回去
- C 因为 ls_free 只能在 finally 里调用



选择题

call 函数里，释放内存为什么写在 finally 里？

A 因为 finally 里的代码跑得更快

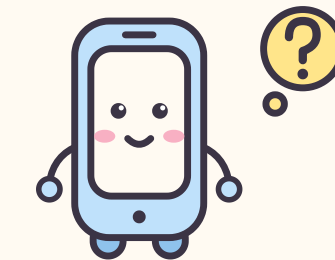
和速度无关。

C 因为 ls_free 只能在 finally 里调用

ls_free 在哪都能调，放在 finally 里是为了保证一定会调到。

B 不管调用成功还是中途抛出异常，申请的内存都必须还回去

finally 一定会执行。写在 try 的末尾的话，一旦中途出错，这几块内存就永远漏掉了。



选择题

计数器在 proto.js 里为什么要先转成 BigInt?

- A 因为 WASM 只认 BigInt
- B JS 的普通数字超过 2 的 53 次方就不精确，64 位计数器要用 BigInt 才能准确拆开
- C 为了让计数器可以是负数



选择题

计数器在 proto.js 里为什么要先转成 BigInt?

A 因为 WASM 只认 BigInt

恰恰相反，我们传给 WASM 的是两个普通的 32 位数字。

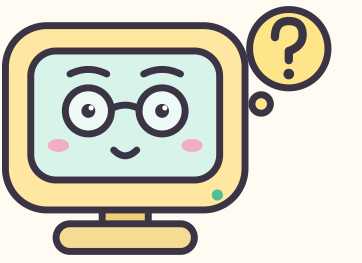
C 为了让计数器可以是负数

计数器从 1 开始，永远不是负数。

B JS 的普通数字超过 2 的 53 次方就不精确，64 位计数器要用 BigInt 才能准确拆开

普通数字是双精度浮点，表示不了所有 64 位整数。用 BigInt 拆出高低两半，再转回普通数字传过去。

想一想



lanshare.wasm 是明文 HTTP 传给小手机的。它被抓包了怎么办？



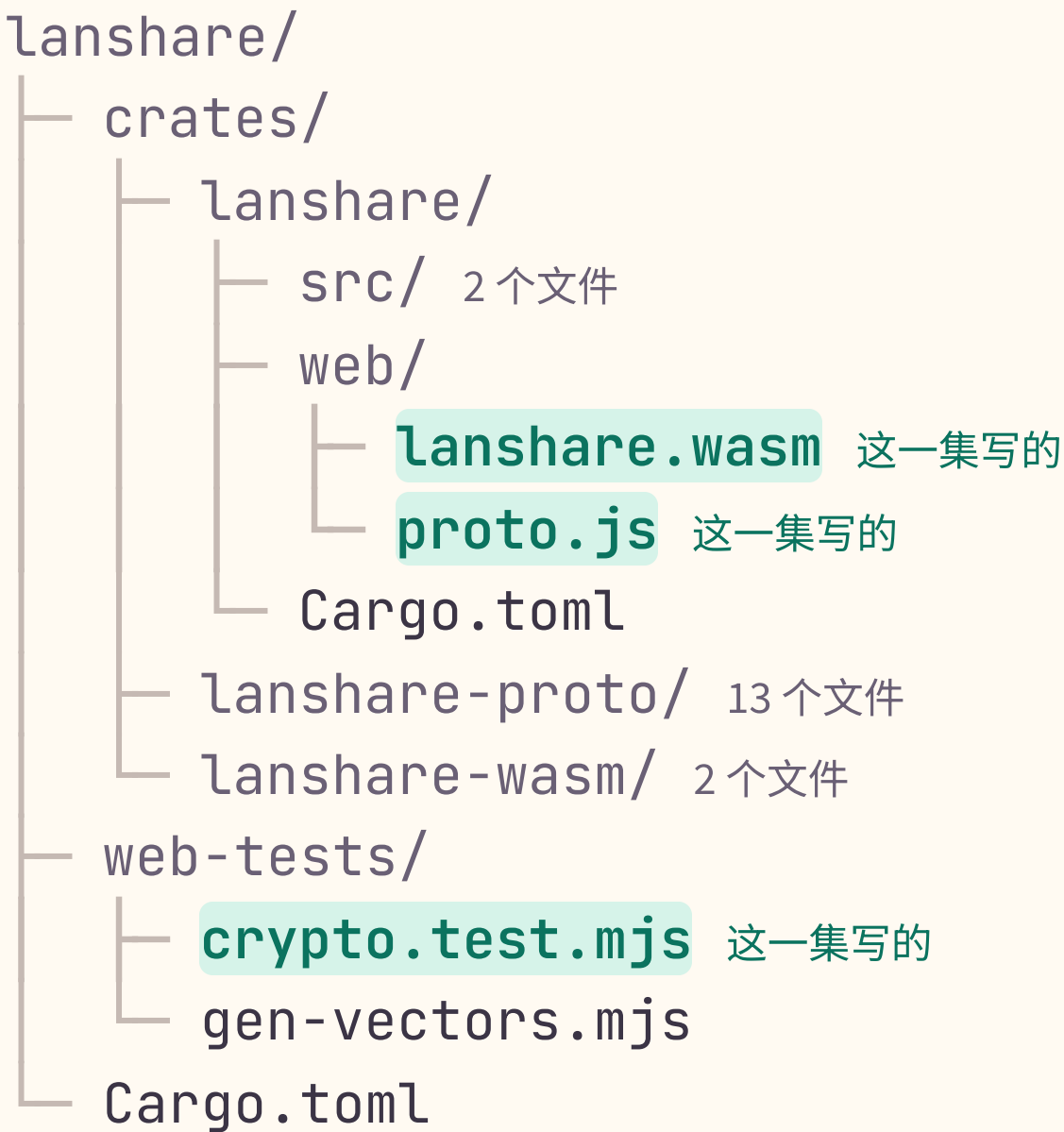
想一想

lanshare.wasm 是明文 HTTP 传给小手机的。它被抓包了怎么办？

要分清两件事。第一，被看到：没关系。lanshare.wasm 本来就是公开的，谁都能直接下载，不用登录。里面只有算法，没有密钥；密钥是老电脑上的程序每次启动时随机生成的，从来不写进这个文件。安全靠的是钥匙，不是把图纸藏起来，密码学里管这叫柯克霍夫原则。第二，被改掉：这才是真正的风险。同一个网络里的捣蛋鬼，比如搞 ARP 欺骗、开恶意热点的人，可以在小手机第一次加载页面时，把 wasm 或者 JS 换成他自己的版本，偷走密钥。在页面里校验 wasm 的哈希也没用：页面本身也走 http，攻击者连校验代码一起换掉就行。这是安全说明里“不防什么”的第一条，也是所有“手机不装 App、只用浏览器”的工具共同的上限。我们防的是窃听，不是篡改。



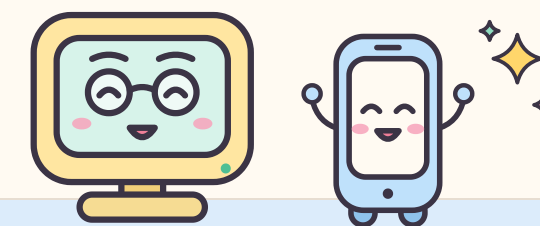
这一集写完了





这一集写了什么

- ✓ loadCrypto: 加载 WASM, 用 env.ls_random 把随机数函数递进去
- ✓ call: 申请 → 拷入 → 调用 → 拷出 → finally 里全部释放
- ✓ 两个坑: 内存扩容后现取视图; 64 位计数器用 BigInt 拆成两半
- ✓ 同一份 OpenSSL 向量: 原生 Rust 和浏览器 WASM 结果完全一致
- ✓ lanshare.wasm 是构建产物; 直接编译会带上本机路径, 发布用 build.ps1 抹掉



下一集

全课完结, 感谢收看